In [266…

```
import kagglehub
import pandas as pd
import torch
import locale
import torch.nn as nn
```

Phase 1: Importing

In [3]:

```
# Download latest version
path = kagglehub.dataset_download("ironwolf437/laptop-price-dataset")

print("Path to dataset files:", path)
```

```
Downloading to C:\Users\ACER\.cache\kagglehub\datasets\ironwolf437\laptop-price-data
set\1.archive...
100%|██████████| 24.5k/24.5k [00:00<00:00, 1.99MB/s]
Extracting files...
Path to dataset files: C:\Users\ACER\.cache\kagglehub\datasets\ironwolf437\laptop-pr
ice-dataset\versions\1
```

Phase 2: Download Dataset

In [33]:

```
dataset = pd.read_csv("./datasets/laptopPrice.csv")

dataset_data = dataset.drop(columns=["Company", "Product", "TypeName", "ScreenResol
dataset_truth = dataset.drop(columns=["Company", "Product", "TypeName", "ScreenReso

print(dataset_data.head())
print(dataset_truth.head())
```

```
   Inches  CPU_Frequency (GHz)  RAM (GB)
0    13.3                  2.3         8
1    13.3                  1.8         8
2    15.6                  2.5         8
3    15.4                  2.7        16
4    13.3                  3.1         8
   Price (Euro)
0       1339.69
1        898.94
2        575.00
3       2537.45
4       1803.60
```

Phase 3: Load Dataset

In [44]:

```
data_sample_count = len(dataset_data["Inches"])
data_feature_count = len(dataset_data.columns)

data_array = []
truth_array = []

# populating data tensor-like array
for sample_index in range(data_sample_count):
```

```
    vector = []

    for feature_index in range(data_feature_count):
        vector.append(dataset_data.iloc[sample_index, feature_index])

    data_array.append(vector)

# data and truth should be the same dimension so i just wing it
truth_feature_count = len(dataset_truth.columns)

# populating truth tensor-like array
for sample_index in range(data_sample_count):
    vector = []

    for feature_index in range(truth_feature_count):
        vector.append(dataset_truth.iloc[sample_index, feature_index])

    truth_array.append(vector)

print(data_array)
print(truth_array)
```

Phase 4: Creating Tensor-Like Data

In [ ]:
```
data = torch.tensor(data_array, dtype=torch.float32)
truth = torch.tensor(truth_array, dtype=torch.float32)
```

```
tensor([[13.3000,  2.3000,  8.0000],
        [13.3000,  1.8000,  8.0000],
        [15.6000,  2.5000,  8.0000],
        ...,
        [14.0000,  1.6000,  2.0000],
        [15.6000,  2.5000,  6.0000],
        [15.6000,  1.6000,  4.0000]]) tensor([[1339.6899],
        [ 898.9400],
        [ 575.0000],
        ...,
        [ 229.0000],
        [ 764.0000],
        [ 369.0000]])
```

Now that i got the data and the truth, i will ignore every variable above and start working with only these two variables so i don't overload my brain.

Now, its time to actually create the equation for training. I will proceed with manual calculation first to make sure i understand.

In [ ]:
```
# defining Weights
print(data.size()[1])
```

```
3
```

in the data, i know that the feature is 3 for each sample so i need to create a weight for each feature plus a bias

In [131…

```
w1 = torch.zeros(1, requires_grad=True)
w2 = torch.zeros(1, requires_grad=True)
w3 = torch.zeros(1, requires_grad=True)
bias = torch.zeros(1, requires_grad=True)
```

okay now the forward pass

In [110…

```
def forward(x):
    return (w1 * x) + (w2 * x) + (w3 * x) + bias
```

Forward pass calculation is always (n_weight * input) + bias so if you got 2 weight then its (weight_1 * input) + (weight_2 * input) + bias

next, is loss function to calculate the error, i will use MSE because its simple and i just know this very well

In [68]:

```
def MSELoss(truth, prediction):
    return ((truth - prediction) ** 2).mean()
```

okay now the *real* training

In [132…

```
epochs = 20000
learning_rate = 0.001

for epoch in range(epochs):
    model_output = forward(data)

    loss = MSELoss(truth, model_output)

    loss.backward()

    with torch.no_grad():
        w1 -= learning_rate * w1.grad
        w2 -= learning_rate * w2.grad
        w3 -= learning_rate * w3.grad
        bias -= learning_rate * bias.grad

    w1.grad.zero_()
    w2.grad.zero_()
    w3.grad.zero_()
    bias.grad.zero_()

    print(f"epoch {epoch} = loss: {loss}")
print()
print("Training Finished!")
```

```
epoch 0 = loss: 1778823.625
epoch 1 = loss: 856094.0
epoch 2 = loss: 750573.9375
epoch 3 = loss: 738226.25
epoch 4 = loss: 736501.9375
epoch 5 = loss: 735989.375
epoch 6 = loss: 735615.25
epoch 7 = loss: 735257.3125
epoch 8 = loss: 734901.6875
epoch 9 = loss: 734546.75
epoch 10 = loss: 734192.1875
epoch 11 = loss: 733838.25
epoch 12 = loss: 733484.6875
epoch 13 = loss: 733131.625
epoch 14 = loss: 732779.0625
epoch 15 = loss: 732426.9375
epoch 16 = loss: 732075.1875
epoch 17 = loss: 731724.0
epoch 18 = loss: 731373.1875
epoch 19 = loss: 731022.9375
epoch 20 = loss: 730673.125
...
epoch 990 = loss: 0.007226541638374329
epoch 991 = loss: 0.007226260378956795
epoch 992 = loss: 0.007225979119539261
epoch 993 = loss: 0.007225697860121727
epoch 994 = loss: 0.007225417532026768
epoch 995 = loss: 0.007225134409964085
epoch 996 = loss: 0.007224854081869125
epoch 997 = loss: 0.007224572356790304
epoch 998 = loss: 0.007224291563034058
epoch 999 = loss: 0.007224010303616524
```

OMG IT WORKED! so i have to do feature scaling to normalize the data so the model doesnt explode in the learning process.

now lets use the model to test it. Because i scale the data in the training process, i need to do the same with the evaluation data.

In [298…

```
screen_inches = 16
CPU_Freq = 2.1
RAM = 16

# feature scale the data
test_data = torch.tensor([[screen_inches, CPU_Freq, RAM]], dtype=torch.float32) / d

price_scaled = linearModel(test_data)

# multiply the data back to the original
price = round((price_scaled * truth_max).item())

price_IDR = price * 20011

locale.setlocale(locale.LC_ALL, 'en_ID.UTF-8')
print(locale.currency(price_IDR, grouping=True))
```

```
# Since the price is in Euro, ill convert the price to IDR
```

```
Rp31.037.061,00
```

In [ ]:
```
print(linearModel.linear1.weight)
print(linearModel.linear1.bias)
```

```
Parameter containing:
tensor([[-0.0646,  0.0131,  0.5993]], requires_grad=True)
Parameter containing:
tensor([0.1529], requires_grad=True)
```